קטע קוד ראשון

```
1. class One {
2. protected:
3. static std::string s;
4. public:
5. One(const std::string& str = s) { s = str; doIt(); }
6. virtual ~One() { cout << "~One: "; doIt(); }
7. virtual void doIt(const string& str = s) {
8. cout << "One::doIt: " << s.c_str() << endl;
9. }
10. };
11.
12. struct Two: public One {
13. Two(const std::string& str = "Boo"): One(str) { doIt(); }
14. One* operator->() { return this; }
15. ~Two() { cout << "~Two: "; doIt(); }
16. virtual void doIt(const string& s = "Two") override {
17. cout << "Two::doIt: " << s.c_str() << endl;
18. }
19. };
20.
21. // the line below is in One.cpp file
22. std::string One::s;
```

תזכורת:

במחלקה `std::string` קיימת המתודה `c_str()` אשר מחזירה את המחרוזת המאוחסנת באובייקט כ- `char*`.

*** הערה לגבי שורות 7 ו-16 בקוד:**

ערך ה-default השונה שניתן בשורות 7 ו-16 הוא חוקי ועובר קומפילציה. זה נחשב רעיון רע לתת ערך default שונה ב-override של פונ' וירטואלית, מטרת השאלה אינה לעודד אתכם לעשות זאת אלא להתייחס למצב שבו מתכנת זוטר בצוות (לא אתם!) עשה זאת (ופוטר!) ואנחנו צריכים עכשיו לדבג לו את הקוד ולהבין מה קורה.
אז מה באמת קורה?
כאשר קוראים לפונקציה doIt ללא פרמטר, הקומפיילר יודע להשלים את הפרמטר בזמן קומפילציה ולהשתמש בערך ברירת המחדל. העובדה שהפונקציה וירטואלית ממשיכה להיות רלבנטית גם אם הפרמטר החסר הושלם ע"י הקומפיילר. בזמן ריצה תמיד מתבצעת קריאה לפונקציה <u>עם פרמטר</u> – בין אם נשלח ובין אם הושלם ע"י הקומפיילר.
השלמה של פרמטר חסר באמצעות ה-default מתבצעת ע"י הקומפיילר <u>בזמן קומפילציה</u> ולכן היא לא מתבצעת לפי סוג האובייקט האמיתי (הידוע בזמן ריצה) אלא בהתאם ל-type של האובייקט המפעיל <u>כפי שידוע לקומפיילר בזמן קומפילציה</u>.
כמו כן אם הקומפיילר לא מבצע קישור דינמי לפונ' – השלמת הפרמטר לברירת המחדל תתבצע לפי הפונ' שאליה הקומפיילר מתכוון לקשר בפועל.

שימו לב: קטעי main אפשריים מופיעים בעמוד הבא!

main A1

```
int main() { .1
Two two;     .2
two.doIt();  .3
two->doIt();  .4
}  .5
```

main A2

```
int main() { .1
Two two;     .2
One* pOne = &two;  .3
pOne->doIt("Foo");  .4
pOne->doIt();  .5
two->doIt();  .6
two->doIt("Moo");  .7
}  .8
```

main A3

```
int main() { .1
Two two("Yoo");  .2
One& one = two;  .3
dynamic_cast<Two&>(one)->doIt();  .4
// One one2 = two;  .5
// one2.doIt();  .6
}  .7
```

קטע קוד B

```
class MyString {
    int size;
    char* str = nullptr;

    // private helpers
    void stealFrom(MyString&& s) {
        cout << "in stealFrom, about to steal: " << s << " into: " << *this << endl;
        size = s.size;
        std::swap(str, s.str);
    }
    MyString(size_t len): size(len), str(new char[size + 1]) {
        str[0] = '\0';
        cout << "in MyString ctor, created: " << *this << endl;
    }

public:
    MyString(const char* s = "") : MyString(strlen(s)) {
        strcpy(str, s);
        cout << "in MyString ctor, setting: " << *this << endl;
    }
    ~MyString() {
        cout << "in ~MyString about to delete: " << *this << endl;
        delete[]str;
    }
    MyString(const MyString& s) : MyString(s.str) {
        cout << "in copy ctor after copying: " << *this << endl;
    }
    MyString(MyString&& s) {
        cout << "in move ctor" << endl;
        stealFrom(std::move(s));
    }
    // note: operator= gets MyString by value - in purpose!
    MyString& operator=(MyString s) {
        cout << "in operator=" << endl;
        stealFrom(std::move(s));
        return *this;
    }
    friend MyString operator+(const MyString& l, const MyString& r) {
        MyString s(l.size + r.size);
        strcpy(s.str, l.str);
        strcpy(s.str + l.size, r.str);
        return s;
    }
    friend ostream& operator<<(ostream& out, const MyString& s) {
        return out << (s.str? s.str : "(null)") << " ("
```

```
48.  << &s << " char* = " << (void*)s.str << ')';
49.  }
50.  };

51.  int main() {
52.      MyString s1 = "Hello";
53.      cout << s1 + " world!" << endl;
54.      MyString s2;
55.      cout << "s2: " << s2 << endl;
56.      s2 = "world " + s1;
57.      cout << "s2: " << s2 << endl;
58.  }
```

תזכורת / הבהרה:

בשורות 18 ו- 26 ישנה קריאה לקונסטרקטור אחר במחלקה, הדבר נקרא constructor delegation.

--- סוף המבחן ---